Javascript Notes

Training session - Javascript Notes

DAY 1 :
05/08/2025

Introduction of Javascript:

- JavaScript is a Scripting language primarily used to create interactive and dynamic content on websites
- JavaScript is typically an interpreted language, meaning code is executed line by line at runtime, offering flexibility in development. It can calculate, manipulate and validate data.

About Browser Engines:

- Chrome - V8 Engine
- Mozilla Firefox - Spider Monkey
- JScript - Internet Explorer

Able to Execute Javascript file without browser using node runtime Environment

➔ To check node version in vs code node -v
➔ To run a code node index.js

JIT Complier:

JIT Complier is a modern compiler in javascript , It combine both interpreter and monitor code line by line at **runtime** to execute the program much better and faster

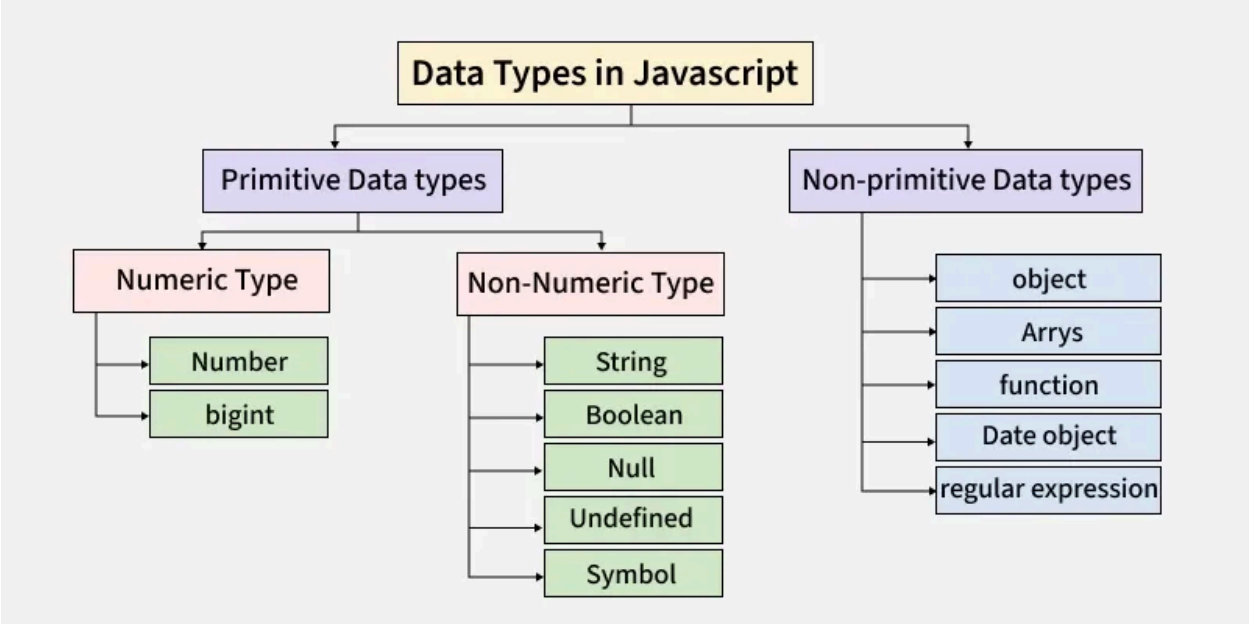
Synchronous and Synchronous:

➔ Javascript is fundamentally Synchronous and single threaded in its execution model, But Javascript also provide Asynchronous operations, which are crucial for a task that involving waiting, data from server, reading files or handling user input

Javascript Statements:

- Declaration Statements (ex : let name =”Logesh”; const age = 23;)
- Assignment Statement (ex : let number = 10)
- Expression(Airthmatic) Statements (ex : x = y + 10; console.log(x))
- Conditional Statements (ex : if, if else, else if, switch)
- Looping Statements (ex : for, while, do.. while)
- Return Statements (ex : used to exit function)
- Break & Continue (ex : Used to terminate & continue a loop)

JavaScript Datatypes:



Primitive data types

- Number (ex : 1, 22.2)
- String (“Logesh”)
- Boolean (true, false)
- Null (absence of object value)
- Undefined (Represents a variable not assigned value)
- Symbol (Symbol data type is used to create objects which will always be unique. these objects can be created using Symbol constructor.) ES6
- BigInt (9007199254740991n) ES2020

Non - Primitive Data Types

- **Object**

  An object in Javascript is an entity having properties and methods. Everything is an object in javascript.

  How to create an object in javascript:

  1.Using Constructor Function to define an object:
  let obj = new Object(); // Create an empty generic object

  // Create a user defined object
  let mycar = new Car();

2.Using Literal notations to define an object:

```
let square = {}; // An empty object
// Here a and b are keys and
// 20 and 30 are values
let circle = {a: 20, b: 30};
```

Example ;

```
// Creating object with the name person
   let person = {
      firstName: "Luiza",
      lastName: "Shaikh",
   };

   // Print the value of object on console
   console.log(person.firstName
```

- **Array**

  Ways to declare a single-dimensional array:

```
// Call it with no arguments
let a = new Array();

// Call it with single numeric argument
let b = new Array(10);

// Explicitly specify two or
// more array elements
let d = new Array(1, 2, 3, "Hello");
```

  Example: Below is an example.

```
let a = new Array();
   let b = new Array(10);
   let d = new Array(1, 2, 3, "Hello");
   console.log("value of a=" + a);
   console.log("value of b" + b);
   console.log("value of d=" + d);
```

- **Function**

```
function greet(name) {
console.log(name);
}
greet(“Logesh”);
```

Example

```
let firstName="Hi, Logesh";
let str="Javascript Functions and this a normal function without
arguments and parameters";
function greetings(){

    console.log(firstName+ " Welcome To "+str );
}
greetings();
```

- Date
- RegExp
- Map
- Set

Boolean Falsy Values

- False
- 0
- 0n or -0.0
- “” (Empty String)
- Null
- Undefined
- NaN

Undefined - Represents a variable not assigned value
Null - absence of object value

Variables:

Variable Declaration

JavaScript provides three ways to declare variables:

Keyword	Scope	Reassignable	Redeclarable	Hoisted	TDZ
`var`	Function	✅ Yes	✅ Yes	✅ Yes	❌ No
`let`	Block	✅ Yes	❌ No	✅ Yes	✅ Yes
`const`	Block	❌ No	❌ No	✅ Yes	✅ Yes

Let’s break each of these down.

1. var

Properties:

- **Function-scoped**
- Can be **redeclared** and **reassigned**
- Hoisted to the top of its scope, but initialized as undefined

2. let

Properties:

- **Block-scoped** (available only within {}, if..)
- Can be **reassigned**, but **not redeclared** in same scope
- Hoisted, but exists in Temporal Dead Zone (TDZ)

3. Const

Properties:

- **Block-scoped (available only within {}, if..)**
- **Cannot be reassigned**
- **Must be initialized** at the time of declaration
- Hoisted + TDZ applies
- const prevents reassignment, not immutability.
- To make truly immutable objects, use:Object.freeze(obj)

Redeclaration & Reassignment Table

Keyword	Redeclaration	Reassignment	Initial Value Required
`var`	✅ Yes	✅ Yes	❌ No
`let`	❌ No	✅ Yes	❌ No
`const`	❌ No	❌ No	✅ Yes

What Is a Variable in JavaScript?

A variable in JavaScript is a named container used to store a value.
let age = 25;

Here, age is the variable name, and 25 is the value it holds.

Conditional Statements:

- if statement
- if...else statement
- if...else if...else statement
- switch statement
- ternary (conditional) operator

1. if Statement

The if statement evaluates a condition inside parentheses. If the condition is true, the block of code inside the curly braces {} runs. If it's false, it skips that block.

Example

```
let x = 20;
if (x % 2 === 0) {
   console.log("Even");
}
```

2. if-else Statement

The if-else statement will perform some action for a specific condition. Here we are using the else statement in which the else statement is written after the if statement and it has no condition in their code block.

Example

```
let age = 25;

if (age >= 18) {
   console.log("Adult")
} else {
   console.log("Not an Adult")
};
```

3. else if Statement

The else if statement in JavaScript allows handling multiple possible conditions and outputs, evaluating more than two options based on whether the conditions are true or false.

Example

```
const x = 0;
if (x > 0) {
   console.log("Positive.");
} else if (x < 0) {
   console.log("Negative.");
} else {
   console.log("Zero.");
};
```

4.Using Switch Statement (JavaScript Switch Case)

The switch statement is a control structure in JavaScript that allows you to execute different blocks of code based on the value of a single expression. It's a cleaner, more readable alternative to multiple if...else if statements when you need to compare one variable against many possible values.

Example

```
const marks = 85;

let Branch;

switch (true) {
   case marks >= 90:
      Branch = "Computer science engineering";
      break;
   case marks >= 80:
      Branch = "Mechanical engineering";
      break;
   case marks >= 70:
      Branch = "Chemical engineering";
      break;
   case marks >= 60:
      Branch = "Electronics and communication";
      break;
   case marks >= 50:
      Branch = "Civil engineering";
      break;
   default:
      Branch = "Bio technology";
      break;
}

console.log(`Student Branch name is : ${Branch}`);
```

5. Using Ternary Operator (?:)

The ternary operator is a compact shorthand for an if...else statement. It is called "ternary" because it takes three operands:

Syntax

```
condition ? expressionIfTrue : expressionIfFalse
```

Example:

```
let age = 21;

const result =
   (age >= 18) ? "You are eligible to vote."(True) : "You are not eligible to vote."(False);

console.log(result);
```

6. Nested if...else

A nested if...else statement is an if...else structure placed inside another if or else block. This allows you to check multiple conditions in a hierarchical or layered way, making complex decision trees possible.

Example

```
let weather = "sunny";
let temp = 25;

if (weather === "sunny") {
   if (temp > 30) {
      console.log("It's a hot day!");
   } else if (temp > 20) {
      console.log("It's a warm day.");
   } else {
      console.log("It's a bit cool today.");
   }
} else if (weather === "rainy") {
   console.log("Don't forget your umbrella!");
} else {
   console.log("Check the weather forecast!");
};
```

Looping Statements (ex : for, while, do.. while)

- for Loop
- while Loop
- do-while Loop
- For- in Loop
- for-of Loop

1. for Loop

The for loop repeats a block of code a specific number of times. It contains initialization, condition, and increment/decrement in one line.

Syntax

```
for (initialization; condition; increment/decrement) {
   // Code to execute
}
```

Example:

```
for (let i = 1; i <= 3; i++) {
   console.log("Count:", i);
}
```

Output

```
Count: 1
Count: 2
Count: 3
```

2. while Loop

The while loop executes as long as the condition is true. It can be thought of as a repeating if statement.

Syntax

```
while (condition) {
   // Code to execute
}
```

while loop

Example

```
let i = 0;
while (i < 3) {
   console.log("Number:", i);
   i++;
}
```

Output
Number: 0
Number: 1
Number: 2

3. do-while Loop

The do-while loop is similar to while loop except it executes the code block at least once before checking the condition.

Syntax

```
do {
   // Code to execute
} while (condition);
```

do-while

Example:

```
let i = 0;
do {
   console.log("Iteration:", i);
   i++;
} while (i < 3);
```

Output
Iteration: 0
Iteration: 1
Iteration: 2

4. For- in Loop

The for...in loop is used to iterate over the properties of an object. It only iterate over keys of an object which have their enumerable property set to “true”.

Syntax

```
for (let key in object) {
   // Code to execute
}
```

```
const obj = { name: "Ashish", age: 25 };
for (let key in obj) {
   console.log(key, ":", obj[key]);
}
```

Output
name : Ashish
age : 25

5. for-of Loop

The for...of loop is used to iterate over iterable objects like arrays, strings, or sets. It directly iterate the value and has more concise syntax than for loop.

Syntax

```
for (let value of iterable) {
   // Code to execute
}
```

Example

```
let a = [1, 2, 3, 4, 5];
for (let val of a) {
   console.log(val);
}
```

Output
1
2
3
4
5

Choosing the Right Loop
Use for loop when the number of iterations is known.
Use while loop when the condition depends on dynamic factors.
Use do-while loop to ensure the block executes at least once.
Use for...in loop to iterate over object properties.
Use for...of loop for iterating through iterable objects.

Functions:

Functions in JavaScript are reusable blocks of code designed to perform specific tasks. They allow you to organize, reuse, and modularize code. It can take inputs, perform actions, and return outputs.

Types of Functions:

- Function Declarations(Named Functions) or Normal Functions
- Function Expression
- Arrow Function (ES6)
- Immediately Invoked Function Expression (IIFE)
- Anonymous Function:
- Callback function

- Pure Function
- Impure Function
- Generator Function
- Async Functions

1.Function Declaration(Named Function) or Normal function

These are defined using the function keyword followed by a name, parameters, and a code block. They are hoisted, meaning they can be called before their declaration in the code.

Example:

```
function sum(x, y) {
   return x + y;
}
console.log(sum(6, 9));
```

2.Function Expression

It is similar to a function declaration without the function name. Function expressions can be stored in a variable assignment.

Example:

```
const mul = function (x, y) {
   return x * y;
};
console.log(mul(4, 5));
```

3. Arrow Functions

Introduced in ES6, these provide a more concise syntax for writing functions, especially for shorter, single-expression functions.

Example:

```
const multiply = (a, b) => a * b;
console.log(multiply(10,20));
```

4.Immediately Invoked Function Expressions (IIFEs)

These are functions that are declared and executed immediately after their definition. They are often used to create a private scope for variables.

Example:

```
(function() {
  console.log("This runs immediately!");
})();
```

5. Anonymous Function

Anonymous functions are functions without a name. They are often used as arguments to other functions.

Example 1:

```
setTimeout(function () {
   console.log("Anonymous function executed!");
}, 1000);
```

Example 2:

```
// Anonymous function assigned to a variable
const greet = function() {
  console.log("Hello from an anonymous function!");
};
greet(); // Output: Hello from an anonymous function!
```

6. Callback function:
A callback function in JavaScript is a function that is passed as an argument to another function

```
// The function that will be passed as a callback
function sayGoodbye() {
  console.log("Goodbye!");
}

// The main function that accepts a callback
function greet(name, callback) {
  console.log(`Hello, ${name}!`);
  // Execute the callback function
  callback();
}

// Call the main function, passing sayGoodbye as the callback
greet("Alice", sayGoodbye);
```

Generator Functions:

These are special functions that can be paused and resumed, allowing for the generation of a sequence of values using the yield keyword. They return an Iterator object.

```
 function* idGenerator() {
     let id = 1;
     while (true) {
       yield id++;
     }
   }
```

Async Functions:

These are used for asynchronous operations and always return a Promise. They allow the use of the await keyword to pause execution until a Promise settles.
JavaScript

```
   async function fetchData() {
     const response = await fetch('some_api_endpoint');
     const data = await response.json();
     return data;
   }
```

ES6+ Features:

Hoisting:

Hoisting is a javascript behaviour able to move function declarations, variable to the top of their scope during the compilation time

Closure:

A closure is a combination of function and lexical scoping , An inner function can able access the outer function variable during the runtime is known as closure

ex:

```
Function example() {
  var name = "Mozilla"; // name is a local variable created by init
  function displayName() {
    // displayName() is the inner function, that forms a closure
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
example();
```

Lexical Scoping:

Lexical scope, also known as static scope, dictates the accessibility of variables and functions based on where they are defined in the source code

Example:

```
function outerFunction() {
  let outerVariable = "I am from the outer scope.";
  function innerFunction() {
     console.log(outerVariable);
     }
   innerFunction(); // Calling innerFunction
}
outerFunction(); // Calling outerFunction
```

Null and Undefined

Undefined

A variable that is declared but not initialized will have the value undefined.

Null:

null is a special value in JavaScript that represents the deliberate absence of any object value

undefined	Null
Indicates a variable that has been declared but not yet assigned a value.	**Represents the intentional absence of any object value.**

Is it possible to use this keyword in the arrow function?

No, Arrow functions in JavaScript do not bind their own this value. Instead, they lexically inherit this from the enclosing scope where they are defined.

Higher Order Function:

Higher-order functions help make your code more reusable and modular by allowing you to work with functions like any other value.

1. Map
The map function is used to transform an array by applying a callback function to each element. It returns a new array.

```
const n = [1, 2, 3, 4, 5];
const square = n.map((num) => num * num);
console.log(square);
```

2. Filter
The filter function is used to create a new array containing elements that satisfy a given condition.

```
const n = [1, 2, 3, 4, 5];
const even = n.filter((num) => num % 2 === 0);
console.log(even);
```

3. Reduce
The reduce function accumulates array elements into a single value based on a callback function.

```
const n = [1, 2, 3, 4, 5];
const sum = n.reduce((acc, curr) => acc + curr, 0);
console.log(sum);
```

Explanation: The callback (acc, curr) => acc + curr adds all elements.
0 is the initial value of the acc.

4. forEach
The forEach function executes a provided function once for each array element.

```
const n = [1, 2, 3];
n.forEach((num) => console.log(num * 2));
```

forEach performs the side effect of printing each element multiplied by 2.
It does not return a new array like map.

Every
The every function checks if all array elements satisfy a condition.

```
const n = [1, 2, 3, 4, 5];
const allPos = n.every((num) => num > 0);
console.log(allPos)
```

The callback (num) => num > 0 checks if all numbers are positive.
It returns true only if all elements pass the condition.

The let keyword
The let variables are mutable, i.e., their values can be changed. It works similarly to the var keyword with some key differences, like scoping, which makes it a better option when compared to var.
Declares a block-scoped local variable. This means the variable is only accessible within the block (e.g., if statement, for loop, or a simple {} block) where it's defined.

```
let n = 10;
{
   let n = 2; // Block-scoped variable
   console.log(n);//2
}
console.log(n);//10
```

The const keyword

const is used to declare variables with a constant value, ensuring the value cannot be reassigned.Declares a block-scoped constant, meaning its value cannot be reassigned after initialization.

```
const PI = 3.14159;
PI = 3; // Error: Assignment to constant variable
```

Temporal Dead Zone

The Temporal Dead Zone (TDZ) in JavaScript refers to a period within a scope where variables declared with let and const exist but cannot be accessed. This period begins at the start of the block scope and ends when the variable's declaration and initialization are encountered during code execution.

Explain call(), apply(), and bind() methods in JavaScript

JavaScript provides the call(), apply(), and bind() methods for setting this context within a function. These methods are especially useful when working with object-oriented code or handling different function contexts.

JavaScript provides three methods for manipulating this keyword in functions: call(), apply(), and bind(). These methods allow you to change the context of this keyword, which can be useful for controlling the behaviour of functions.

1. call()
The call() method invokes a function with a specified this value and arguments provided individually.

```
const person1 = { name: "Alice" };
const person2 = { name: "Bob" };

function greet(message) {
  console.log(`${message}, ${this.name}!`);
}

greet.call(person1, "Hello"); // Outputs: Hello, Alice!
greet.call(person2, "Hi"); // Outputs: Hi, Bob!
```

2. apply()
The apply() method is similar to call(), but it accepts arguments as an array.JavaScript

```
const numbers = [1, 2, 3];

function sum(a, b, c) {
  console.log(a + b + c);
}

sum.apply(null, numbers); // Outputs: 6 (null indicates no specific 'this' context needed)
```

3. bind()
The bind() method creates a new function that, when called, has its this keyword set to the provided value, with a given sequence of arguments preceding any provided when the new function is called.

```
const user = { username: "devuser" };

function getUsername() {
  return this.username;
}

const boundGetUsername = getUsername.bind(user);
console.log(boundGetUsername()); // Outputs: devuser
```

Prototype and Prototype Chaining:

Prototype:
This mechanism is fundamental to how JavaScript implements inheritance. When you try to access a property or method on an object, if it's not found directly on that object, JavaScript looks for it on its prototype.

In JavaScript, every object has an internal property called [[Prototype]] (exposed as __proto__ in some environments, though Object.getPrototypeOf() is the standard way to access it). This prototype is either null or references another object.

Example

```
function Animal(name) {
  this.name = name;
}

// Add a method to the Animal's prototype
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound.`);
};

const dog = new Animal('Buddy');
dog.speak(); // Output: Buddy makes a sound.
```

Prototype Chaining in JavaScript

Prototype chaining occurs when an object's prototype itself has a prototype, forming a chain of objects. When a property or method is accessed, JavaScript traverses this chain, searching for the property or method on each object in the chain until it finds it or reaches the end of the chain (which is null).

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal walks.");
  }
};

const rabbit = Object.create(animal); // rabbit's prototype is 'animal'
rabbit.jumps = true;
const babyRabbit = Object.create(rabbit); // babyRabbit's prototype is 'rabbit'
console.log(babyRabbit.eats); // Output: true (found on 'animal' through 'rabbit')
babyRabbit.walk(); // Output: Animal walks. (found on 'animal' through 'rabbit')
console.log(babyRabbit.jumps); // Output: true (found on 'rabbit')
```

This Keyword In Javascript:

The 'this keyword' in JavaScript refers to the (Current Instance) object to which it belongs

```
const person = {
    name: "GeeksforGeeks",
    greet() {
        return `Welcome To, ${this.name}`;
    }
};
console.log(person.greet());
```

Array:

Creating an Array
Using an array literal is the easiest way to create a JavaScript Array.

Syntax:

```
const array_name = [item1, item2, ...];
```

Example:
```
const cars = ["Saab", "Volvo", "BMW"];
```

Using the JavaScript Keyword new
The following example also creates an Array, and assigns values to it:

Example
```
const cars = new Array("Saab", "Volvo", "BMW");
```

Changing an Array Element
This statement changes the value of the first element in cars like overwrite that index:

Example
```
const cars = ["Saab", "Volvo", "BMW"];
cars[0] = "Opel";
```

Converting an Array to a String
The JavaScript method toString() converts an array to a string of (comma separated) array values.

Example
```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits.toString();
```

Objects:

What are the possible ways to create objects in JavaScript?

There are many ways to create objects in JavaScript, as mentioned below:

- **Object literal syntax:**

The object literal syntax (or object initializer) is a comma-separated set of name-value pairs wrapped in curly braces.

```
var object = {
name: "Sudheer,"
age: 34,
};
```

Object literal property values can be of any data type, including array, function, and nested object.

Note: This is one of the easiest ways to create an object, and it's most commonly used for creating simple, ad hoc objects.

- **Object constructor:**

The simplest way to create an empty object is using the `Object` constructor. Currently this approach is not recommended.

```
var object = new Object();
```

There `Object()` is a built-in constructor function, so the "new" keyword is not required for creating plain objects. The above code snippet can be rewritten as:

```
var object = Object();
```

However, it `Object()` can be used to either create a plain object or convert a given value into its corresponding object wrapper, whereas it `new Object()` is specifically used to explicitly create a new object instance.

- **Objects' create method:**

The `create` method of Object is used to create a new object by passing the specified prototype object and properties as arguments, i.e., this pattern is helpful to create new objects based on existing objects. In other words, this is useful for setting up **prototypal inheritance**. The second argument is optional, and it is used to create properties on a newly created object.

The following code creates a new empty object whose prototype is null.

```
var object = Object.create(null);
```

The following example creates an object along with additional new properties.

```
let vehicle = {
wheels: "4",
fuelType: "Gasoline",
color: "Green",
};
let carProps = {
type: {
value: "Volkswagen",
},
model: {
value: "Golf",
},
};

var car = Object.create(vehicle, carProps);
console.log(car);
```

- **Function constructor:**

In this approach, create any function and apply the new operator to create object instances. This was the main way to do constructor-based OOP before ES6 classes.

```
function Person(name) {
this.name = name;
this.age = 21;
}
var object = new Person("Sudheer");
```

- **Function constructor with prototype:**

This is similar to a function constructor, but it uses a prototype for their properties and methods. Using prototype means you're sharing methods/properties across instances, which saves memory and improves performance.

```
function Person() {}
Person.prototype.name = "Sudheer";
var object = new Person();
```

This is equivalent to creating an instance with `Object.create` a method with a function prototype and then calling that function with an instance and parameters as arguments.

```
function func(x, y, z) {
this.x = x;
this.y = y;
this.z = z;
}

var instance = new func(1, 2, 3);
```

(OR)

```
function func(x, y, z) {
this.x = x;
this.y = y;
this.z = z;
}
// Create a new instance using function prototype.
var newInstance = Object.create(func.prototype)

// Call the function
var result = func.call(newInstance, 1, 2, 3),

// If the result is a non-null object then use it; otherwise, just use the new instance.
console.log(result && typeof result === 'object' ? result : newInstance);
```

- **Object's assign method:**

The `Object.assign` method is used to copy all the properties from one or more source objects and store them into a target object. This is mainly used for cloning and merging
The following code creates a new staff object by copying properties of his working company and the car he owns.

```
const orgObject = { company: "XYZ Corp" };
const carObject = { name: "Toyota" };
const staff = Object.assign({}, orgObject, carObject);
```

- **ES6 Class syntax:**

ES6 introduces class feature to create objects. This is syntactic sugar over the prototype-based system.

```
class Person {
constructor(name) {
this.name = name;
}
}

var object = new Person("Sudheer");
```

- **Singleton pattern:**

A singleton is an object which can only be instantiated one time. Repeated calls to its constructor return the same instance. This way one can ensure that they don't accidentally create multiple instances.
Singleton with Closure (Classic JS Pattern)

```
const Singleton = (function () {
let instance;

function createInstance() {
return { name: "Sudheer" };
}

return {
getInstance: function () {
if (!instance) {
instance = createInstance();
}
return instance;
```

```
}
};
})();

// Usage
const obj1 = Singleton.getInstance();
const obj2 = Singleton.getInstance();

console.log(obj1 === obj2); // true
```

In modern JavaScript applications, singletons are commonly implemented using ES6 modules for their built-in caching behavior, or closures for encapsulated state management.

Mock Notes

Introduction

First of all, I'm really glad to express myself to you. I'm Logesh, currently studying Javascript and also eager to learn React, Node, MongoDb, Express and Version Control(Git). I am presently working at Mitrahsoft Solutions Private Limited as a Software Trainee.

Project - Healthcare Appointment Organizer

What is Javascript?

JavaScript is a scripting language primarily used to create interactive and dynamic content on websites and also javascript used for both client side scripting and server side Development.

Hoisting:

Hoisting is a javascript behaviour able to move function declarations, variable to the top of their scope during the compilation time

```
console.log(x);
var x=10;
```

Closure:

A closure is a combination of function and lexical scoping (Environment) , In This, An inner function can able access the outer function variable during the runtime is known as closure , It "remembers" its lexical environment

Ex:

```
function example() {
  Let name = "Logesh"; // name is a local variable created by init
  function displayName() {
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
example();
```

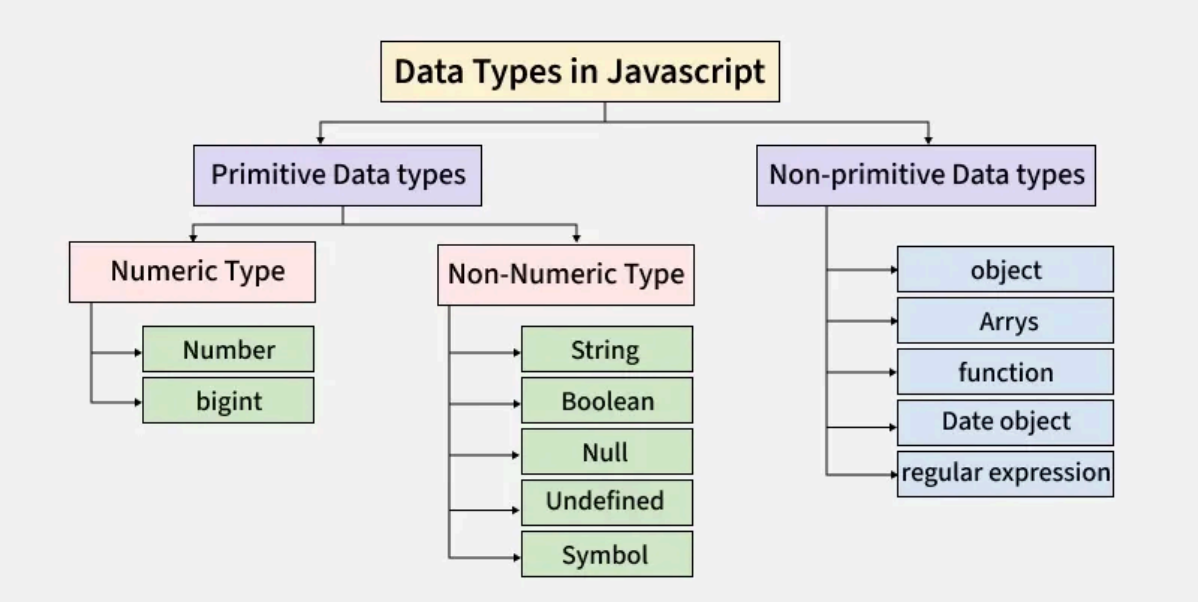
Lexical Scoping:

Lexical scope, also known as static scope, dictates the accessibility of variables and functions based on where they are defined in the source code

Example:

```
function outerFunction() {
  let outerVariable = "I am from the outer scope.";
  function innerFunction() {
     console.log(outerVariable);
     }
   innerFunction(); // Calling innerFunction
}
outerFunction(); // Calling outerFunction
```

Datatypes:



Callback function:

In JavaScript is function that is passed as an argument to another function is called a callback function.

And i can be executed inside its function whenever it is called

```
function  A(n){
        console.log("A - Higher Order Function for Below Call Back function");
        n(); // Executes the function passed as 'n'
}

function B(){
        console.log("B"); //
}
A(B); // Call function A, passing B as the callback
```

Null and Undefined

Undefined

A variable that is declared but not initialized will have the value undefined.

Null:

null is a special value in JavaScript that represents the deliberate absence of any object value

undefined	Null
Indicates a variable that has been declared but not yet assigned a value.	**Represents the intentional absence of any object value.**

Is it possible to use this keyword in the arrow function?

No, Arrow functions in JavaScript do not bind their own this value. Instead, they lexically inherit this from the enclosing scope where they are defined.

Higher Order Function:

Higher-order functions help make your code more reusable and modular by allowing you to work with functions like any other value.

1. Map
The map function is used to transform an array by applying a callback function to each element. It returns a new array.

```
const n = [1, 2, 3, 4, 5];
const square = n.map((num) => num * num);
console.log(square);
```

2. Filter
The filter function is used to create a new array containing elements that satisfy a given condition.

```
const n = [1, 2, 3, 4, 5];
const even = n.filter((num) => num % 2 === 0);
console.log(even);
```

3. Reduce

The reduce function accumulates array elements into a single value based on a callback function.

```
const n = [1, 2, 3, 4, 5];
const sum = n.reduce((acc, curr) => acc + curr, 0);
console.log(sum);
```

The callback (acc, curr) => acc + curr adds all elements.
0 is the initial value of the acc.

4. forEach
The forEach function executes a provided function once for each array element.

```
const n = [1, 2, 3];
n.forEach((num) => console.log(num * 2));
```

forEach performs the side effect of printing each element multiplied by 2.
It does not return a new array like map.

Every
The every function checks if all array elements satisfy a condition.

```
const n = [1, 2, 3, 4, 5];
const allPos = n.every((num) => num > 0);
console.log(allPos)
```

The callback (num) => num > 0 checks if all numbers are positive.
It returns true only if all elements pass the condition.

Explain call(), apply(), and bind() methods in JavaScript

JavaScript provides the call(), apply(), and bind() methods for setting this context within a function. These methods are especially useful when working with object-oriented code or handling different function contexts.

JavaScript provides three methods for manipulating the this keyword in functions: call(), apply(), and bind(). These methods allow you to change the context of the this keyword, which can be useful for controlling the behaviour of functions.

1. call()

The call() method invokes a function with a specified this value and arguments provided individually.

```
const person1 = { name: "Alice" };
const person2 = { name: "Bob" };

function greet(message) {
  console.log(`${message}, ${this.name}!`);
}

greet.call(person1, "Hello"); // Outputs: Hello, Alice!
greet.call(person2, "Hi"); // Outputs: Hi, Bob!
```

2. apply()
The apply() method is similar to call(), but it accepts arguments as an array.JavaScript

```
const numbers = [1, 2, 3];

function sum(a, b, c) {
  console.log(a + b + c);
}

sum.apply(null, numbers); // Outputs: 6 (null indicates no specific 'this' context needed)
```

3. bind()
The bind() method creates a new function that, when called, has its this keyword set to the provided value, with a given sequence of arguments preceding any provided when the new function is called.
JavaScript

```
const user = { username: "devuser" };

function getUsername() {
  return this.username;
}

const boundGetUsername = getUsername.bind(user);
console.log(boundGetUsername()); // Outputs: devuser
```

Prototype and Prototype Chaining:

Prototype:
This mechanism is fundamental to how JavaScript implements inheritance. When you try to access a property or method on an object, if it's not found directly on that object, JavaScript looks for it on its prototype.

In JavaScript, every object has an internal property called [[Prototype]] (exposed as __proto__ in some environments, though Object.getPrototypeOf() is the standard way to access it). This prototype is either null or references another object.

Example

```
function Animal(name) {
  this.name = name;
}

// Add a method to the Animal's prototype
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound.`);
};

const dog = new Animal('Buddy');
dog.speak(); // Output: Buddy makes a sound.
```

Prototype Chaining in JavaScript

Prototype chaining occurs when an object's prototype itself has a prototype, forming a chain of objects. When a property or method is accessed, JavaScript traverses this chain, searching for the property or method on each object in the chain until it finds it or reaches the end of the chain (which is null).

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal walks.");
  }
};

const rabbit = Object.create(animal); // rabbit's prototype is 'animal'
rabbit.jumps = true;
const babyRabbit = Object.create(rabbit); // babyRabbit's prototype is 'rabbit'
console.log(babyRabbit.eats); // Output: true (found on 'animal' through 'rabbit')
babyRabbit.walk(); // Output: Animal walks. (found on 'animal' through 'rabbit')
console.log(babyRabbit.jumps); // Output: true (found on 'rabbit')
```

This Keyword In Javascript:

The 'this keyword' in JavaScript refers to the (Current Instance) object to which it belongs

```
const person = {
    name: "GeeksforGeeks",
    greet() {
        return `Welcome To, ${this.name}`;
    }
};
console.log(person.greet());
```

ES6+ Features:

The let keyword

The let variables are mutable, i.e., their values can be changed. It works similarly to the var keyword with some key differences, like scoping, which makes it a better option when compared to var.

Declares a block-scoped local variable. This means the variable is only accessible within the block (e.g., if statement, for loop, or a simple {} block) where it's defined.

```
let n = 10;
{
    let n = 2; // Block-scoped variable
    console.log(n);//2
}
console.log(n);//10
```

The const keyword

const is used to declare variables with a constant value, ensuring the value cannot be reassigned.Declares a block-scoped constant, meaning its value cannot be reassigned after initialization.

```
 const PI = 3.14159;
PI = 3; // Error: Assignment to constant variable
```

Temporal Dead Zone

The Temporal Dead Zone (TDZ) in JavaScript refers to a period within a scope where variables declared with let and const exist but cannot be accessed. This period begins at the start of the block scope and ends when the variable's declaration and initialization are encountered during code execution.

Arrow Functions

Arrow functions provide a concise syntax for writing function expressions and automatically bind this to the surrounding context. Concise Syntax: Arrow functions provide a shorter syntax for writing function expressions, especially for single-line functions or callbacks.

```
// Traditional function
function add(a, b) {
  return a + b;
 }

// Arrow function
const add = (a, b) => a + b;
```

Copy of Mock Notes

Introduction

First of all, I'm really glad to express myself to you. I'm Logesh ,currently working as a MERN Stack Developer with over 2+ years of professional experience in design and developing web based applications and also worked in both client side designing and server side scripting. I am presently working at Mitrahsoft Solutions Private Limited.

Project - Healthcare Appointment Organizer

What is Javascript?

JavaScript is a scripting language primarily used to create interactive and dynamic content on websites and also javascript used for both client side scripting and server side Development.

Hoisting:

Hoisting is a javascript behaviour able to move function declarations, variable to the top of their scope during the compilation time

Closure:

A closure is a combination of function and lexical scoping , An inner function can able access the outer function variable during the runtime is known as closure

ex:

```
Function example() {
  var name = "Logesh"; // name is a local variable created by init
  function displayName() {
    // displayName() is the inner function, that forms a closure
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
example();
```

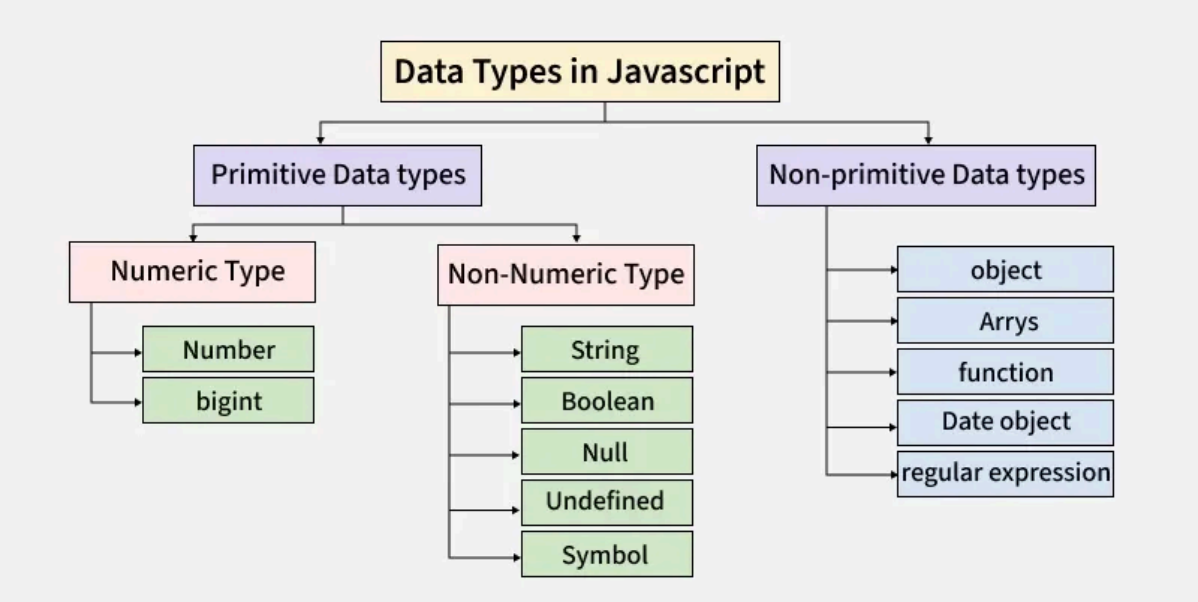
Lexical Scoping:

Lexical scope, also known as static scope, dictates the accessibility of variables and functions based on where they are defined in the source code

Example:

```
function outerFunction() {
  let outerVariable = "I am from the outer scope.";
  function innerFunction() {
     console.log(outerVariable);
     }
   innerFunction(); // Calling innerFunction
}
outerFunction(); // Calling outerFunction
```

Datatypes:



Callback function:
A callback function in JavaScript is a function that is passed as an argument to another function

```
function  A(n){
        console.log("A");
        n();
}
A(B);
function B(){
        console.log("B");
}
```

Null and Undefined

Undefined
A variable that is declared but not initialized will have the value undefined.

Null:
null is a special value in JavaScript that represents the deliberate absence of any object value

undefined	Null
Indicates a variable that has been declared but not yet assigned a value.	**Represents the intentional absence of any object value.**

Is it possible to use this keyword in the arrow function?

No, Arrow functions in JavaScript do not bind their own this value. Instead, they lexically inherit this from the enclosing scope where they are defined.

Higher Order Function:

Higher-order functions help make your code more reusable and modular by allowing you to work with functions like any other value.

1. Map
The map function is used to transform an array by applying a callback function to each element. It returns a new array.

```
const n = [1, 2, 3, 4, 5];
const square = n.map((num) => num * num);
console.log(square);
```

2. Filter
The filter function is used to create a new array containing elements that satisfy a given condition.

```
const n = [1, 2, 3, 4, 5];
const even = n.filter((num) => num % 2 === 0);
console.log(even);
```

3. Reduce

The reduce function accumulates array elements into a single value based on a callback function.

```
const n = [1, 2, 3, 4, 5];
const sum = n.reduce((acc, curr) => acc + curr, 0);
console.log(sum);
```

The callback (acc, curr) => acc + curr adds all elements.
0 is the initial value of the acc.

4. forEach
The forEach function executes a provided function once for each array element.

```
const n = [1, 2, 3];
n.forEach((num) => console.log(num * 2));
```

forEach performs the side effect of printing each element multiplied by 2.
It does not return a new array like map.

Every
The every function checks if all array elements satisfy a condition.

```
const n = [1, 2, 3, 4, 5];
const allPos = n.every((num) => num > 0);
console.log(allPos)
```

The callback (num) => num > 0 checks if all numbers are positive.
It returns true only if all elements pass the condition.

Explain call(), apply(), and bind() methods in JavaScript

JavaScript provides the call(), apply(), and bind() methods for setting this context within a function. These methods are especially useful when working with object-oriented code or handling different function contexts.

JavaScript provides three methods for manipulating the this keyword in functions: call(), apply(), and bind(). These methods allow you to change the context of the this keyword, which can be useful for controlling the behaviour of functions.

1. call()

The call() method invokes a function with a specified this value and arguments provided individually.

```
const person1 = { name: "Alice" };
const person2 = { name: "Bob" };

function greet(message) {
  console.log(`${message}, ${this.name}!`);
}

greet.call(person1, "Hello"); // Outputs: Hello, Alice!
greet.call(person2, "Hi"); // Outputs: Hi, Bob!
```

2. apply()
The apply() method is similar to call(), but it accepts arguments as an array.JavaScript

```
const numbers = [1, 2, 3];

function sum(a, b, c) {
  console.log(a + b + c);
}

sum.apply(null, numbers); // Outputs: 6 (null indicates no specific 'this' context needed)
```

3. bind()
The bind() method creates a new function that, when called, has its this keyword set to the provided value, with a given sequence of arguments preceding any provided when the new function is called.
JavaScript

```
const user = { username: "devuser" };

function getUsername() {
  return this.username;
}

const boundGetUsername = getUsername.bind(user);
console.log(boundGetUsername()); // Outputs: devuser
```

Prototype and Prototype Chaining:

Prototype:
This mechanism is fundamental to how JavaScript implements inheritance. When you try to access a property or method on an object, if it's not found directly on that object, JavaScript looks for it on its prototype.

In JavaScript, every object has an internal property called [[Prototype]] (exposed as __proto__ in some environments, though Object.getPrototypeOf() is the standard way to access it). This prototype is either null or references another object.

Example

```
function Animal(name) {
  this.name = name;
}

// Add a method to the Animal's prototype
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound.`);
};

const dog = new Animal('Buddy');
dog.speak(); // Output: Buddy makes a sound.
```

Prototype Chaining in JavaScript

Prototype chaining occurs when an object's prototype itself has a prototype, forming a chain of objects. When a property or method is accessed, JavaScript traverses this chain, searching for the property or method on each object in the chain until it finds it or reaches the end of the chain (which is null).

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal walks.");
  }
};

const rabbit = Object.create(animal); // rabbit's prototype is 'animal'
rabbit.jumps = true;
const babyRabbit = Object.create(rabbit); // babyRabbit's prototype is 'rabbit'
console.log(babyRabbit.eats); // Output: true (found on 'animal' through 'rabbit')
babyRabbit.walk(); // Output: Animal walks. (found on 'animal' through 'rabbit')
console.log(babyRabbit.jumps); // Output: true (found on 'rabbit')
```

This Keyword In Javascript:

The 'this keyword' in JavaScript refers to the (Current Instance) object to which it belongs

```
const person = {
    name: "GeeksforGeeks",
    greet() {
        return `Welcome To, ${this.name}`;
    }
};
console.log(person.greet());
```

ES6+ Features:

The let keyword

The let variables are mutable, i.e., their values can be changed. It works similarly to the var keyword with some key differences, like scoping, which makes it a better option when compared to var.

Declares a block-scoped local variable. This means the variable is only accessible within the block (e.g., if statement, for loop, or a simple {} block) where it's defined.

```
let n = 10;
{
    let n = 2; // Block-scoped variable
    console.log(n);//2
}
console.log(n);//10
```

The const keyword

const is used to declare variables with a constant value, ensuring the value cannot be reassigned.Declares a block-scoped constant, meaning its value cannot be reassigned after initialization.

```
 const PI = 3.14159;
PI = 3; // Error: Assignment to constant variable
```

Temporal Dead Zone

The Temporal Dead Zone (TDZ) in JavaScript refers to a period within a scope where variables declared with let and const exist but cannot be accessed. This period begins at the start of the block scope and ends when the variable's declaration and initialization are encountered during code execution.

Arrow Functions

Arrow functions provide a concise syntax for writing function expressions and automatically bind this to the surrounding context. Concise Syntax: Arrow functions provide a shorter syntax for writing function expressions, especially for single-line functions or callbacks.

```
// Traditional function
function add(a, b) {
  return a + b;
 }

// Arrow function
const add = (a, b) => a + b;
```

Mock Practice Notes

```
//1.Hoisting

console.log(x);//undefined (var Have a default value - undefined)
var x=10;
//var function scope

console.log(y);//Reference Error (Let Does not have a default value)
let y=20;
//Block Scope

console.log(z);//Reference Error (Const Does not have a default value)
const z=30;
//Block Scope
```

Hourly status

Plan for a Day 19/08/2025

- Plan to Revise Core Javascript Concepts
- Plan to learn Document Object Model (DOM)

EOD Status Update 13/08/2025

- Revising Core Javascript Concepts
- Attended the JavaScript class on Objects.
- Learned about call(), apply(), bind().
- Learn about the object creation, type of object creation, accessing and modify.
- learned the difference between Slice, Splice Methods.
- learning about Pure , Impure and Currying Function.
- The given task was completed and send in txt file to the respective person.

links

https://flexboxfroggy.com/

https://uiverse.io/

need to learn

Higher order function - hof